

Chapter 3.1

User Interfaces

Contents

- Basic Widgets
- Stateless and Stateful Widget
- Design Systems
 - Cupertino Widgets
 - Material Widgets
- Layout
 - Single-child
 - Multi-child
- Sliver Widgets
- Performance best practices

Basic Widgets

Widgets

Widgets are UI components of your app. They define the :

- **A**ppearance
- **L**ayout, and
- **B**ehavior of a specific part of your app's interface

A widget represents a **single** element (e.g. button, text field, image) or even a **complex layout** container.

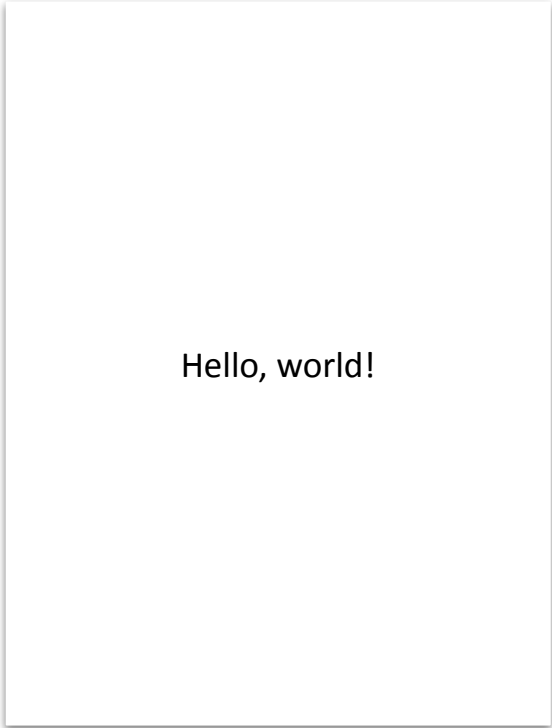
Widgets are **nested hierarchically**, forming a tree-like structure that defines the overall UI of your app.

Widgets

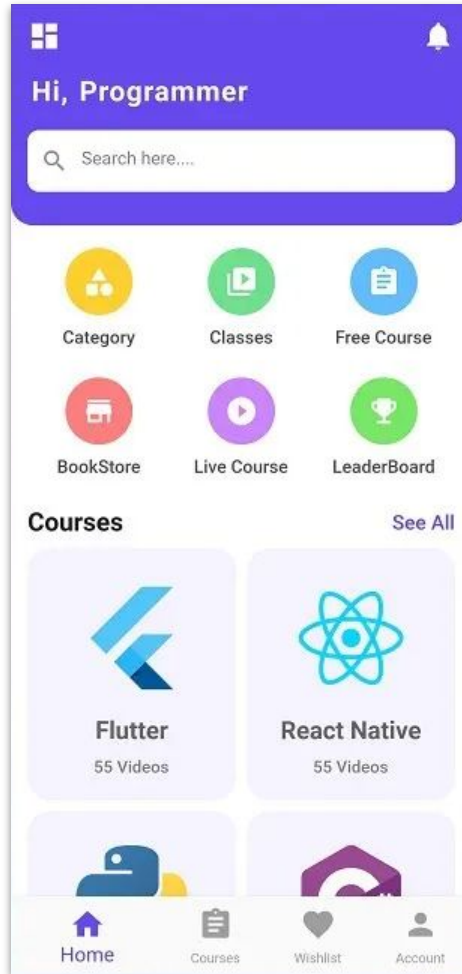
The minimum Flutter app with a text widget:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Hello, world!',
        textDirection: TextDirection.ltr,
      ),
    ),
  );
}
```



Hello, world!



Column

- Container
 - Column
 - Text
 - Search Bar
- Column
 - Row
 - Row
- Text
- Wrap
 - ListView
 - Card

Widget - Characteristics

Customizable:

- Most widgets offer various properties that you can adjust to customize their appearance and behavior.
- This allows you to create unique and personalized UIs.

Widget - Characteristics

Reusable:

- Widgets are designed to be reused throughout your app, promoting code efficiency and consistency.

```
ElevatedButton( onPressed: () {  
    // Handle button press },  
    child: Text('Button 1'), ),  
    // ... other parts of your app
```

```
CustomButton( text: 'Custom Button',  
    onPressed: () { // Handle button press },  
    color: Colors.green, ),
```


Basic widgets

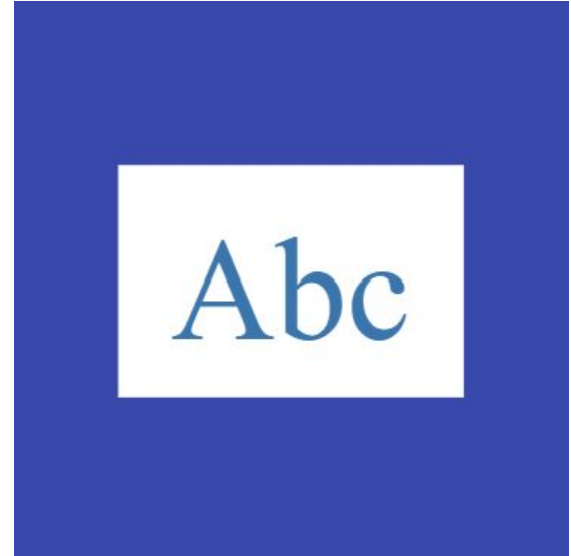
1. Text
2. TextField
3. ElevatedButton
4. Image
5. Row/Column
6. Scaffold
7. Placeholder
8. Icon
9. AppBar

Text

Text

Displays text content.

A run of text with a single style.



Text

The Text widget displays a string of text with single style.

The style argument is optional.

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    Text(
      'Hello, world!',
      textDirection: TextDirection.ltr,
    ),
  );
}
```

Text

```
Container(  
  width: 100,  
  decoration: BoxDecoration(border: Border.all()),  
  child: Text(  
    overflow: TextOverflow.ellipsis,  
    'Hello $_name, how are you?',  
    textDirection: TextDirection.ltr,),  
  )  
)
```

Hello Ali, ...

Text

```
const Text.rich(  
  TextSpan(  
    text: 'Hello', // default text style  
    children: <TextSpan>[  
      TextSpan(  
        text: ' beautiful ',  
        style: TextStyle(  
          fontStyle: FontStyle.italic)  
        ),  
      TextSpan(  
        text: 'world',  
        style: TextStyle(  
          fontWeight: FontWeight.bold)  
        ),  
    ],  
  ),  
)
```

Hello, *beautiful* **world**

TextField

A text field lets the user enter text, either with hardware keyboard or with an onscreen keyboard.



Enter a search term

TextField

- InputDecoration includes label, icon, inline hint text, and error text.
- To remove the decoration entirely, set the decoration to null.

```
TextField(  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    hintText: 'Enter a search term',  
  ), ),
```



TextField

- You can get input value from a TextField in 3 ways:
 - `onChanged()`
 - `onSubmitted()`
 - Controller

Reading Values

A common way to read a value from a TextField is to use the **onSubmitted** callback.

This callback is applied to the text field's current value when the user finishes editing.

It can be used to manually move focus to another input widget when a user finishes with the currently focused input widget.

Reading Values

The default behavior of getting value is an invocation of **onChanged**.

The text field calls the **onChanged** callback whenever the user changes the text in the field.

TextField

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Email Address',  
    hintText: 'Enter your email',  
    icon: Icon(Icons.email),  
  ),  
  keyboardType: TextInputType.emailAddress,  
  
  // Reference to a TextEditingController object  
  controller: _emailController,  
  onChanged: (text) => print('Email: $text'),  
),
```



Email Address

Text Fields

Selecting the appropriate keyboard type, you can significantly improve the user's input experience and reduce errors.



text

Normal text keyboard



emailAddress

Normal text keyboard
with the @ character



number

Basic number keypad

phone

Phone-style keypad

TextField

The `TextEditingController` class is used to manage the text input in a `TextField`.

It allows you to:

1. Set the initial text: Set the default text that appears in the `TextField`.
2. Retrieve the current text: Get the text that the user has entered.
3. Listen to text changes: Respond to changes in the text as the user types.
4. Clear the text: Remove all text from the `TextField`.

TextField

```
class _MyTextFieldAppState extends State<MyTextFieldApp> {  
  final TextEditingController _textController = TextEditingController();
```

```
  void _submitText() {  
    String text = _textController.text;  
    // Do something with the text, e.g., print it to the console  
    print("Entered text: $text");  
  }  
  ...
```

Retrieve the current text

```
  TextField(  
    controller: _textController,  
    hintText: 'Enter your text here',),
```

Assign controller

```
  ElevatedButton(  
    onPressed: _submitText,  
    child: Text('Submit'),),
```

TextField

Clear the text.

```
_textController.clear();
```

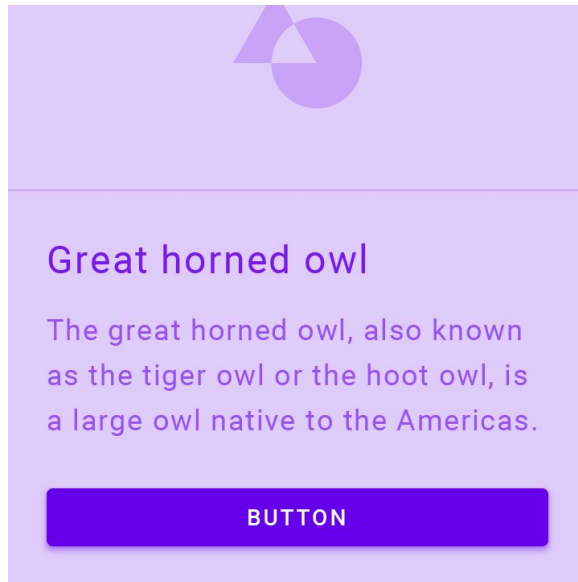
Dispose the Controller when the widget is removed to avoid memory leaks.

```
@override  
void dispose() {  
  _textController.dispose();  
  super.dispose();  
}
```

ElevatedButton

Creates buttons for user interaction.

A filled button whose material elevates when pressed.

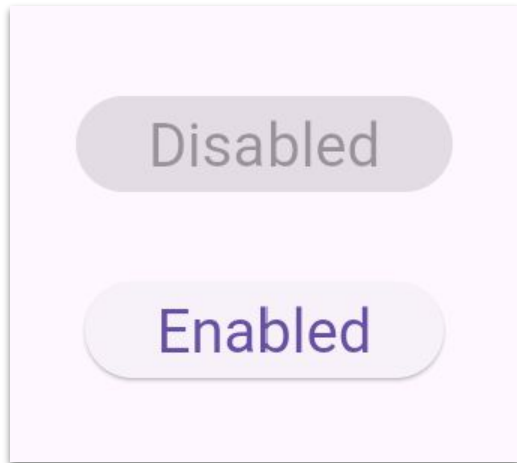


ElevatedButton

```
ElevatedButton(  
  style: style,  
  onPressed: null,  
  child: const Text('Disabled'),  
),
```

```
const SizedBox(height: 30),
```

```
ElevatedButton(  
  style: style,  
  onPressed: () {},  
  child: const Text('Enabled'),  
),
```



Elevated Button

Text Button

Outlined Button



IconButton



FloatingActionButton

Image

Image: Displays images.

Ways to display images:

1. Assets folder
2. Explicit device's directory
3. Network



Image

1. Assets folder

Step 1:

Create a folder named '**assets**' in the project **root directory**

Step 2:

Copy all images to the assets folder.

Reference: <https://api.flutter.dev/flutter/widgets/Image/Image.asset.html>

Image

1. Assets folder

Step 3:

Open the project's **pubspec.yaml** file and enter the image file names

```
flutter:  
  assets:  
    - images/cat.png  
    - images/2x/cat.png  
    - images/3.5x/cat.png
```

Image

1. Assets folder

Step 4:

Write code to display an image from asset

```
Image.asset('images/cat.png')
```

Image

2. Explicit device's directory

Displays an image obtained from a specific location, e.g. from a photo app

```
Image.file(selectedImages, ),
```

Reference: <https://api.flutter.dev/flutter/widgets/Image/Image.file.html>

Image

3. Network

To load images from a network URL:

```
Image.network('https://example/image_name.jpg')
```

Note: Reading a file from network involve an asynchronous task, which cannot provide a result immediately when it is started

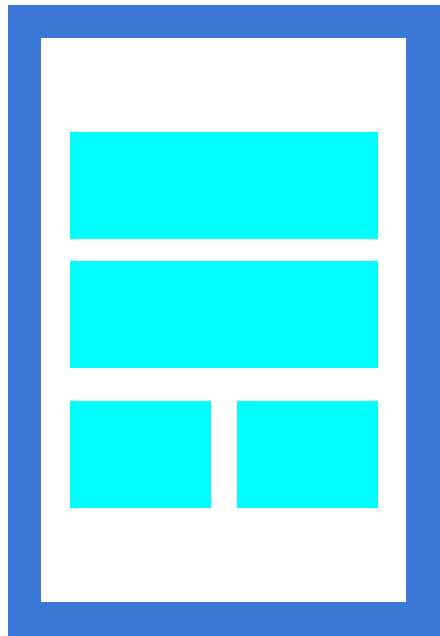
Image

Format image using

```
Image.asset(  
  file: file_name,  
  height: 100,  
  width: 100,  
  fit: BoxFit.fill,  
  alignment: Alignment.center,  
),
```

Row/Column

Arranges widgets horizontally (column) or vertically (row) for layout purposes.



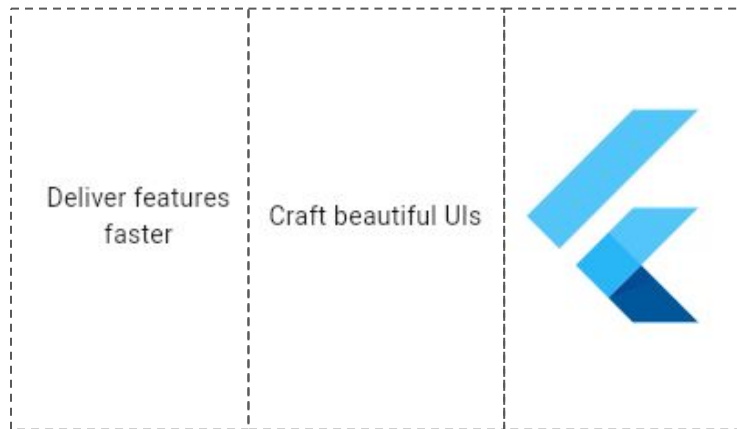
Row/Column

```
const Column(  
  children: <Widget>[  
    Text('Deliver features faster'),  
    Text('Craft beautiful UIs'),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```



Row/Column

```
const Row(  
  children: <Widget>[  
    Expanded(  
      child: Text('Deliver features faster',  
        textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: Text('Craft beautiful UIs',  
        textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```



Icon

Icon: Displays icons.

```
const Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: <Widget>[  
    Icon(  
      Icons.favorite,  
      color: Colors.pink,  
      size: 24.0,  
      semanticLabel: 'Text to announce in accessibility modes',  
    ),  
    Icon(  
      Icons.audiotrack,  
      color: Colors.green,  
      size: 30.0,  
    ),  
    ...
```



Scaffold

Implements the basic Material Design visual layout structure.

It provides APIs for showing drawers, snack bars, and bottom sheets.

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('Sample Scaffold'),  
  ),  
  body: Center(...),  
);
```

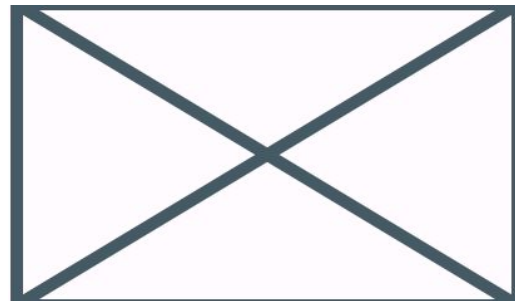


Placeholder

It represents a place where other widgets will be added later.

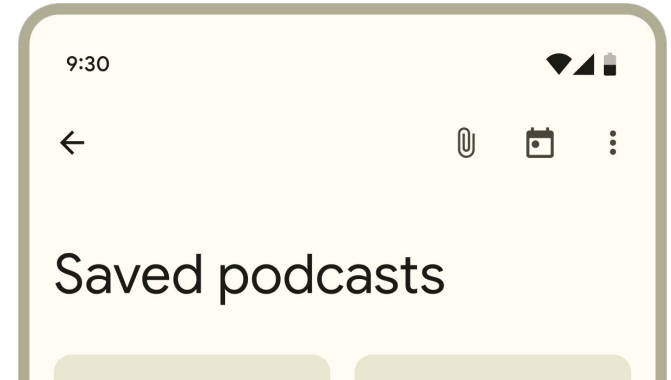
It is useful during development to indicate that the interface is not yet complete.

```
const Placeholder(  
  fallbackHeight: 30.0,  
  fallbackWidth: 100.0,  
  strokeWidth: 5.0,  
)
```



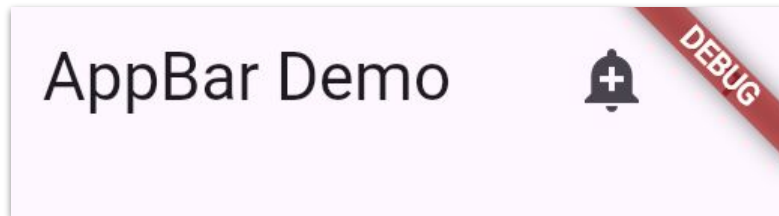
AppBar

Container that displays content and actions at the top of a screen.



AppBar

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('AppBar Demo'),  
      actions: <Widget>[  
        IconButton(  
          icon: const Icon(Icons.add_alert),  
          tooltip: 'Show SnackBar',  
          onPressed: null,  
        ),  
      ],  
    ),  
  );  
}
```



Stateless vs Stateful Widget

Widgets - Stateless vs Stateful

Stateless widgets

Simpler and have a fixed appearance and behavior

Don't hold any internal state

Stateful widgets

Maintain their own state, which react to user interactions or changes in data

Manage state through a State object

Stateless Widget

```
class GreenFrog extends StatelessWidget {  
  const GreenFrog({ super.key });  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: const Color(0xFF2DBD3A));  
  }  
}
```

Stateless Widget

```
class Frog extends StatelessWidget {  
  const Frog({  
    super.key,  
    this.color = const Color(0xFF2DBD3A),  
    this.child,  
  });  
  
  final Color color;  
  final Widget? child;  
  
  @override  
  Widget build(BuildContext context) {  
    return ColoredBox(color: color, child: child);  
  }  
}
```

Stateful Widget

```
class YellowBird extends StatefulWidget {  
  const YellowBird({ super.key });  
  
  @override  
  State<YellowBird> createState() => _YellowBirdState();  
}  
  
class _YellowBirdState extends State<YellowBird> {  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: const Color(0xFFFFE306));  
  }  
}
```

Stateful Widget

```
class Bird extends StatefulWidget {  
  const Bird({  
    super.key,  
    this.color = const Color(0xFFFFE306),  
    this.child,  
  });  
  
  final Color color;  
  final Widget? child;  
  
  @override  
  State<Bird> createState() =>  
    _BirdState();  
}
```

```
class _BirdState extends State<Bird> {  
  double _size = 1.0;  
  
  void grow() {  
    setState(() { _size += 0.1; });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: widget.color,  
      transform:  
        Matrix4.diagonal3Values(_size,  
                                _size, 1.0),  
      child: widget.child,  
    );  
  }  
}
```

Design Systems

Design Systems

Cupertino

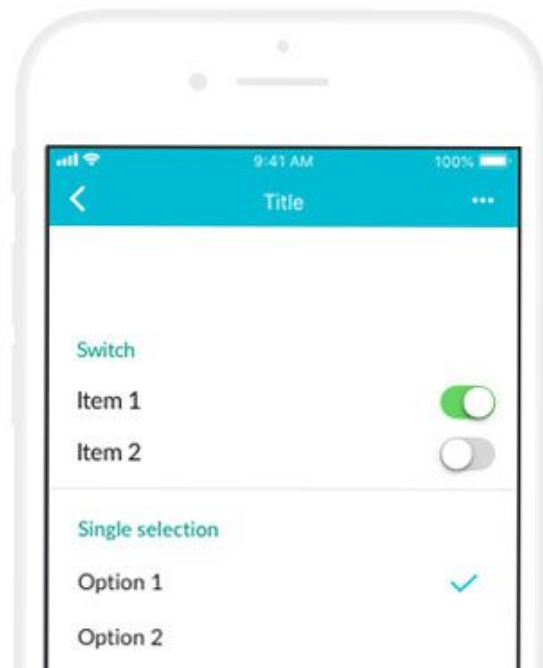
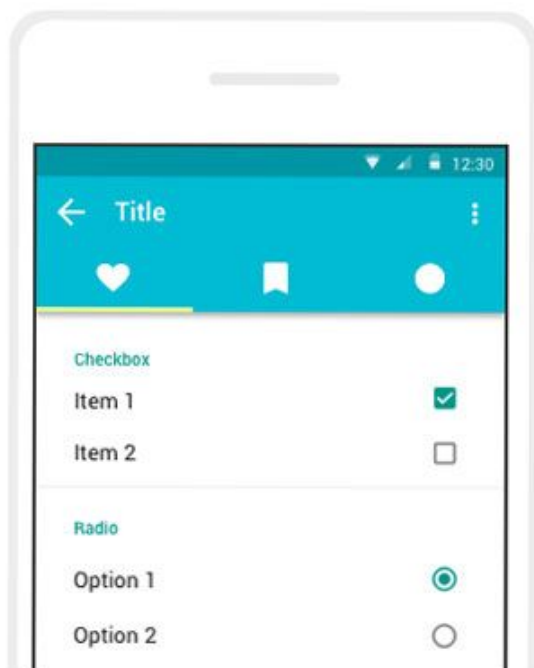
Widgets align with Apple's Human Interface Guidelines for iOS and macOS



Material Components

Visual, behavioral, and motion-rich widgets implementing the Material 3 design specification for Android, web, and desktop applications.



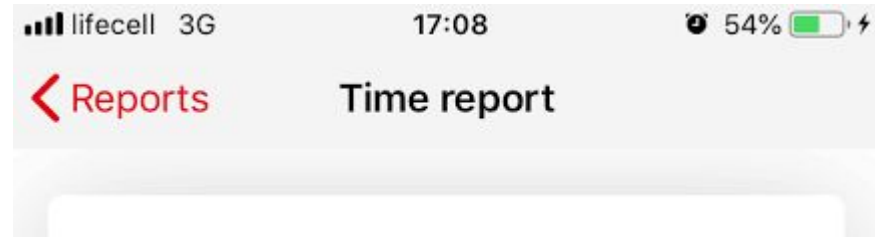


Cupertino Widgets

CupertinoPageScaffold: Provides a basic structure for a Cupertino-style app.

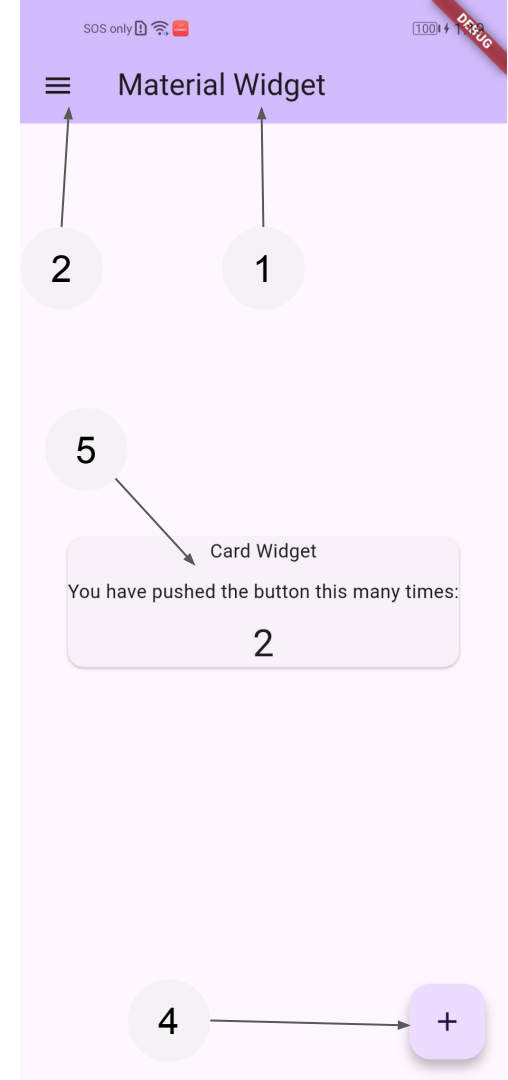
CupertinoNavigationBar: Creates a navigation bar for Cupertino-style apps.

CupertinoButton: Creates a button with a Cupertino-style appearance.



Material Design Widgets

1. AppBar: Creates an app bar at the top of the screen.
2. Drawer: Creates a side menu that slides in from the side.
3. Scaffold: Provides a basic structure for a material design app.
4. FloatingActionButton: Creates a circular button that floats above the content.
5. Card: Displays a rectangular card with a shadow.



Layout

Layout

- The core of Flutter's layout mechanism is widgets
- A layout is used to build more complex widgets



CALL



ROUTE



SHARE

Layout

- A layout is a widget.
- Two main categories of layout widgets:



Single-child



Multi-child

Single-child layout widgets

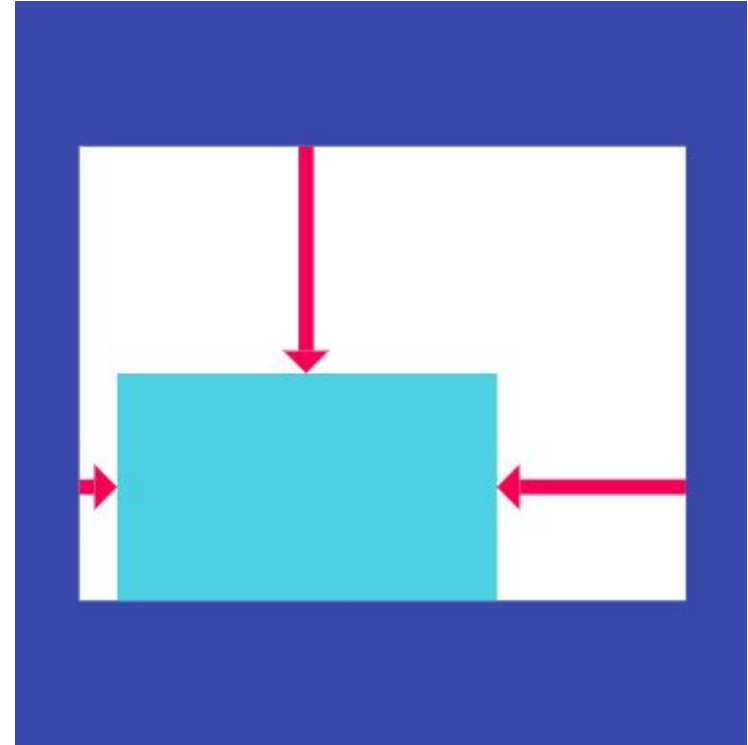
Single-child layout widgets

- These widgets can only hold one child widget.
- They are commonly used for tasks like:
 - Align
 - AspectRatio
 - Center
 - ConstrainedBox
 - Container
 - Padding
 - SizedBox



Align

A widget that aligns its child within itself and optionally sizes itself based on the child's size.



Align

```
Center(  
  child: Container(  
    height: 120.0,  
    width: 120.0,  
    color: Colors.blue[50],  
    child: const Align(  
      alignment: Alignment.topRight,  
      child: FlutterLogo(  
        size: 60,  
      ),  
    ),  
  ),  
)
```

Alignment.topRight



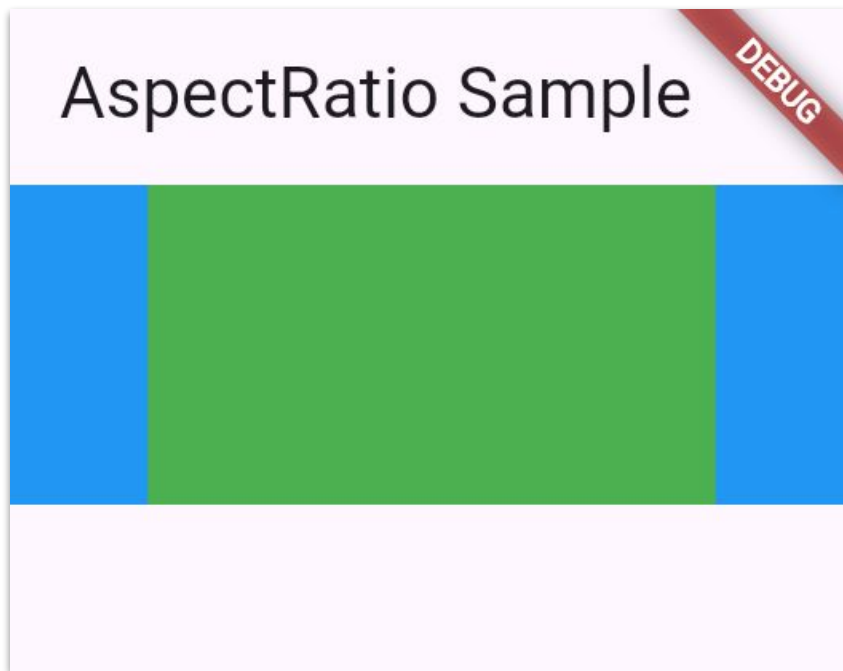
AspectRatio

A widget that attempts to size the child to a specific aspect ratio.



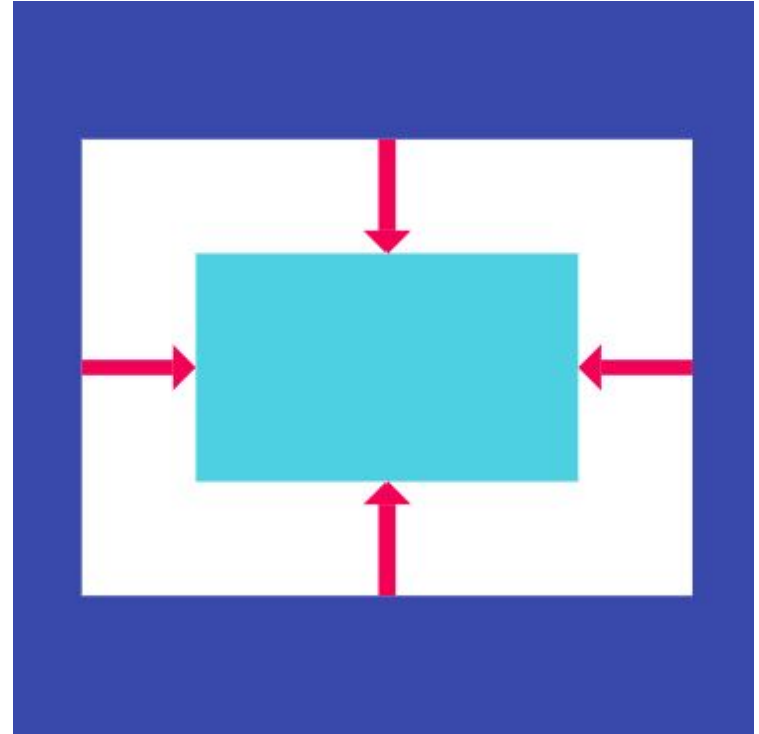
AspectRatio

```
Container(  
  color: Colors.blue,  
  alignment: Alignment.center,  
  width: double.infinity,  
  height: 100.0,  
  child: AspectRatio(  
    aspectRatio: 16 / 9,  
    child: Container(  
      color: Colors.green,  
    ),  
  ),  
)
```



Center

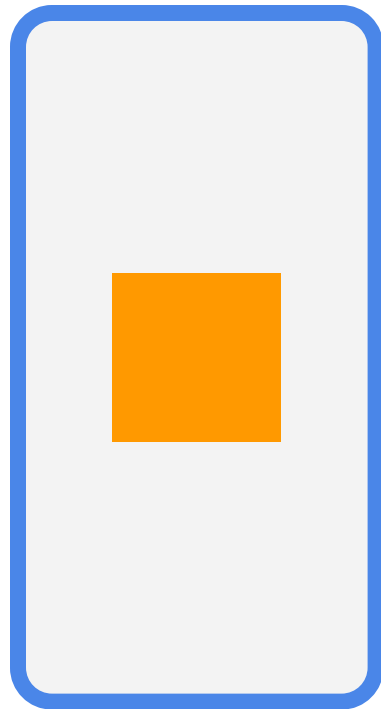
Alignment block that centers its child within itself.



Container

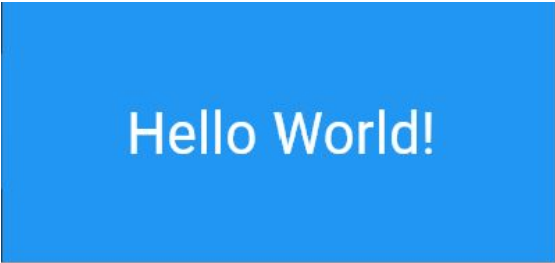
Groups other widgets and provides styling options like padding, margins, and borders.

```
Center(  
  child: Container(  
    margin: const EdgeInsets.all(10.0),  
    color: Colors.amber[600],  
    width: 48.0,  
    height: 48.0,  
  ),  
)
```



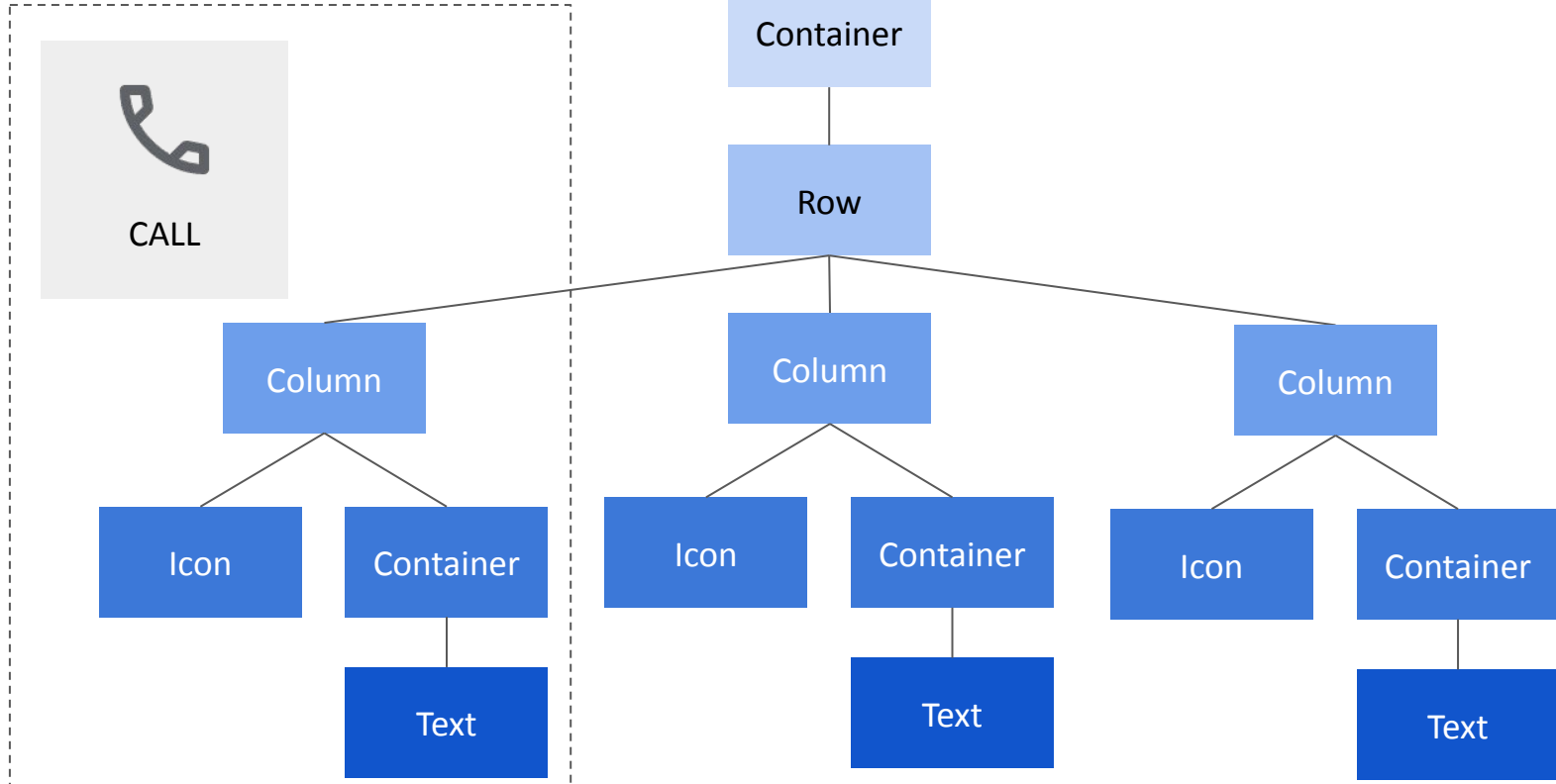
Container

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.blue,  
  padding: EdgeInsets.all(20.0),  
  alignment: Alignment.center,  
  child: Text(  
    "Hello World!",  
    style: TextStyle(  
      color: Colors.white,  
      fontSize: 20.0,  
      ...
```



Hello World!

Container



Padding

A widget that insets its child by the given padding.

```
const Card(  
  child: Padding(  
    padding: EdgeInsets.all(16.0),  
    child: Text('Hello World!'),  
  ),  
)
```



SizedBox

A box with a specified size. If given a child, this widget forces it to have a specific width and height.

```
const SizedBox(  
  width: 200.0,  
  height: 300.0,  
  child: Card(child: Text('Hello World!')), //optional  
)
```

Multi-child layout widgets

Multi-child layout widgets

These widgets can accommodate multiple child widgets and arrange them in different ways.

Common Multi-child Widgets:

- Row/Column
- GridView
- ListView
- Stack
- Wrap



Multi-child layout widgets

Key Features:

1. Layout Management

These widgets handle the arrangement and positioning of their child widgets within the available space.

2. Alignment

You can control how child widgets are aligned within the parent widget using properties like **mainAxisAlignment** (for Row and Column) or **alignment** (for Stack).

Multi-child layout widgets

Key Features:

3. Spacing (Flutter 3.27)

Properties like `mainAxisAlignment` (Row/Column) and `spacing` (Row/Column) allow you to control the spacing between child widgets.

4. Sizing

Some multi-child widgets offer options to control the size of their child widgets, such as **`mainAxisSize`** (Row/Column) or `flex` (Flex).

Row and Column

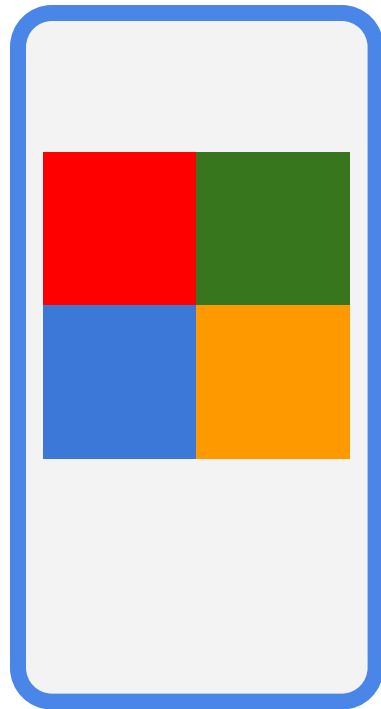
```
Row(  
    children:[  
        Text('Widget 1'),  
        Text('Widget 2'),  
        Text('Widget 3'),  
    ],  
)
```

```
Column(  
    children:[  
        Text('Widget 1'),  
        Text('Widget 2'),  
        Text('Widget 3'),  
    ],  
)
```


GridView

A scrollable, 2D array of widgets.

```
GridView.count(  
  crossAxisCount: 2,  
  children: [  
    Container(color: Colors.red,),  
    Container(color: Colors.green,),  
    Container(color: Colors.blue,),  
    Container(color: Colors.orange,),  
  ],  
),
```

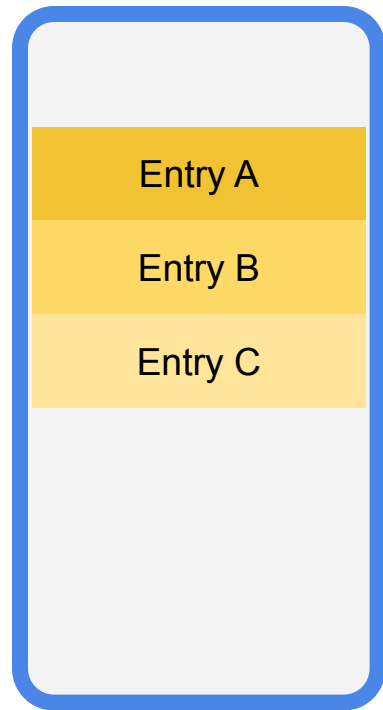


ListView

A scrollable list of widgets arranged linearly.

Suitable for lists that contain only a few items.

```
body: ListView(  
  children: [  
    Container(height: 50, color: Colors.amber[600],  
      child: const Center(child: Text('Entry A')),  
    ),  
    Container(height: 50, color: Colors.amber[500],  
      child: const Center(child: Text('Entry B')),  
    ),  
    Container(height: 50, color: Colors.amber[100],  
      child: const Center(child: Text('Entry C')),  
    ),  
  ],  
),
```

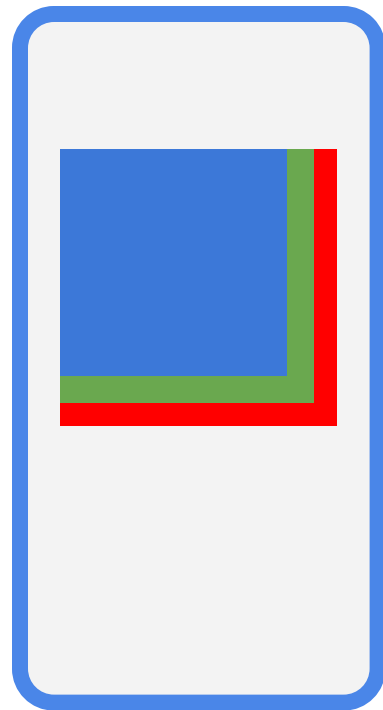


Stack

A widget that positions its children relative to the edges of its box.

It is useful if you want to overlap several children in a simple way.

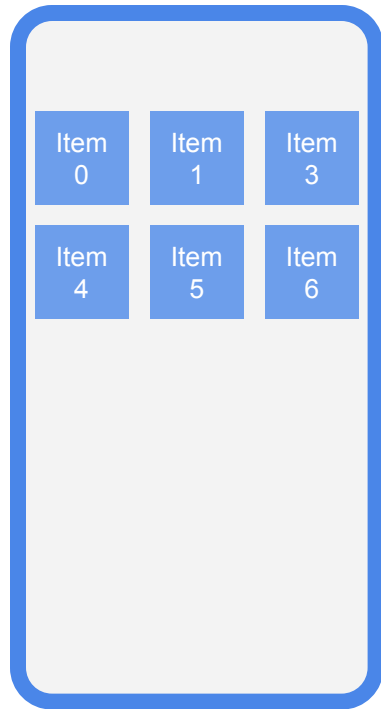
```
Stack(  
  children: <Widget>[  
    Container(width: 100, height: 100,  
      color: Colors.red,),  
    Container(width: 90, height: 90,  
      color: Colors.green,),  
    Container(width: 80, height: 80,  
      color: Colors.blue,),  
  ],  
)
```



Wrap

A widget that displays its children in multiple horizontal or vertical runs.

```
Wrap(  
  children: List.generate(6, (index) {  
    return Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Container( width: 100, height: 100,  
        color: Colors.lightBlue,  
        child: Center(  
          child: Text('Item $index',)),),  
    );  
  }),  
,
```



Form Widgets

TextField: Allows users to enter text.

Checkbox: Creates a checkbox for user input.

RadioListTile: Creates a radio button for user input.

DropDownButtonFormField: Creates a dropdown menu for selecting a value.

The image shows a mobile application interface for a 'Signup Form'. The top status bar is blue with the text 'Operator' and a Wi-Fi icon on the left, and '5:00 p.m.' and a battery icon on the right. Below the status bar is a blue header bar with a white back arrow on the left and the title 'Signup Form' in white. The main content area is white and contains four text input fields with labels 'Full Name', 'Email', 'Password', and 'Confirm Password'. The 'Confirm Password' field has a green checkmark icon to its right. Below the input fields is a toggle switch for the text 'I have read and accept the terms of service.'. At the bottom of the form is a blue button with the text 'Signup' in white.

Layout Widgets

Flex: Provides more control over the layout of its children using flex factors.

Wrap: Wraps its children to the next line when the available space runs out.

CustomScrollView: Allows you to create custom scrolling experiences.

SliverAppBar: Creates an app bar that can expand, collapse, and display different content as the user scrolls.

Other Widgets

FutureBuilder: Builds a widget based on the result of an asynchronous computation.

StreamBuilder: Builds a widget based on the latest data from a stream.

GestureDetector: Detects gestures on a child widget.

Hero: Creates a hero animation between two widgets in different screens.

Sliver Widgets

Sliver widgets

- Sliver widgets are the building blocks for creating **custom scrolling experiences**.
- They provide a more **granular approach** to scroll view construction compared to standard widgets like *ListView* or *GridView*.

Sliver widgets

Concepts

- Unlike regular box widgets, slivers are designed specifically for **scrollable areas**.
- They can **dynamically adjust their size and shape** based on the viewport and scrolling position.
- This allows for creating intricate **scrolling effects** like parallax effects, expanding headers, and custom layouts within a scrollable view.

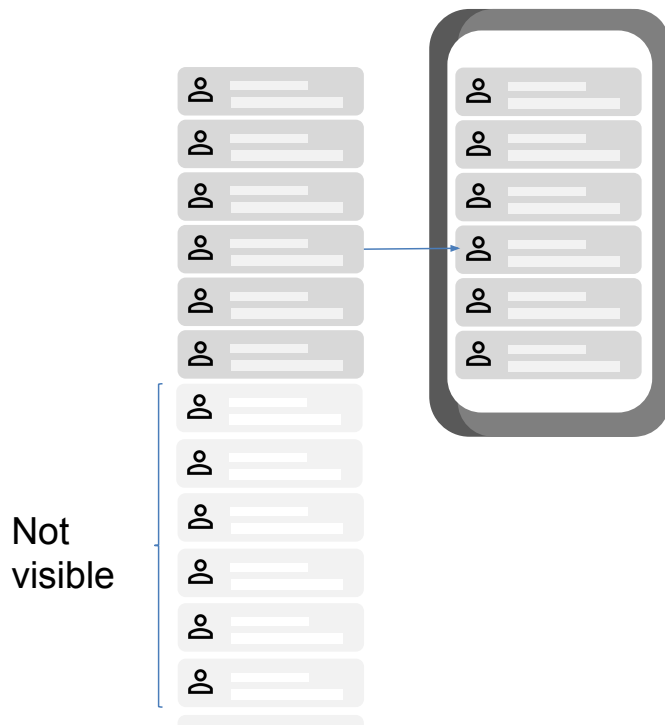
Sliver widgets

Benefits

- **Customization:** Sliver widgets offer fine-grained control over scrolling behavior and appearance.
- **Performance:** For large data sets, slivers can be more efficient than standard scrollable widgets.
- **Flexibility:** You can combine different slivers to create unique and engaging scrolling experiences.

Sliver widgets

All records are loaded to the ListView



Load records that are visible to the user using SilverList



Sliver widgets

Using Sliver Widgets

- Sliver widgets are typically used with the CustomScrollView widget.
- CustomScrollView takes a list of slivers as its slivers property, allowing you to combine different slivers to achieve your desired layout.

Sliver widgets

Common Sliver Widgets:

- **SliverAppBar**: Creates an app bar that can expand, collapse, and display different content as the user scrolls.
- **SliverList**: Renders a list of widgets vertically, similar to `ListView`, but with more flexibility in sizing and performance optimizations for large lists.
- **SliverGrid**: Renders a grid of widgets in a similar way to `GridView`, but within the context of a scrollable area.
- **SliverToBoxAdapter**: Embeds a regular box widget (like a `Text` or a `Container`) within the scrollable area.
- **SliverFixedExtentList**: Similar to `SliverList` but forces all its children to have the same height, improving rendering efficiency for uniform lists.

Performance Best Practices

Performance Best Practices

1. Minimize Widget Rebuilds

Avoid repetitive and costly work in **build()** methods since it can be invoked frequently when ancestor widgets rebuild.

Use **const** for **immutable widgets**: This tells Flutter that the widget's properties won't change, leading to optimizations.

Utilize **const constructors**: Use const constructors for widgets that don't require state.

Performance Best Practices

2. Optimize Layout

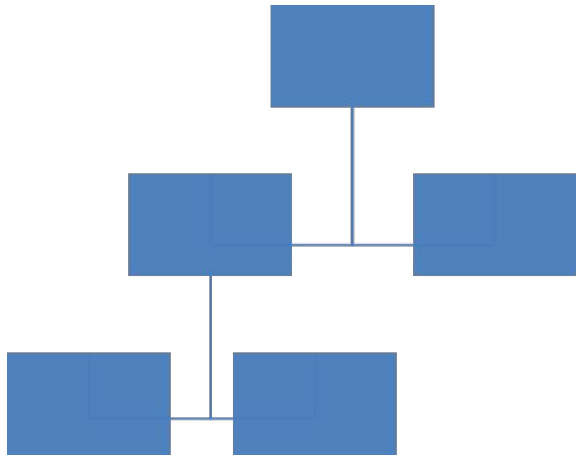
Avoid unnecessary nesting: Keep your widget tree as flat as possible to reduce rendering time.

Use **LayoutBuilder**: For dynamic layouts that depend on available space, use LayoutBuilder.

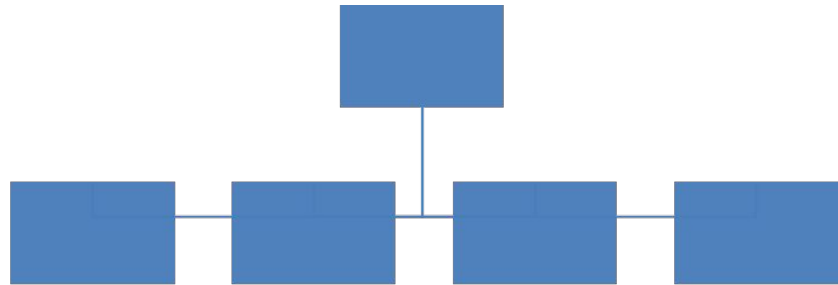
Performance Best Practices

2. Optimize Layout

Which layout is better in term of performance?



(A)



(B)

Performance Best Practices

3. Efficient Image Loading

Use `Image.network`: For network images, use `Image.network` with appropriate caching and loading strategies.

Optimize Image Size: Resize images to the appropriate size for your app.

Use efficient image formats like WebP, which is a raster graphics file format to replace JPEG, PNG and GIF.

Performance Best Practices

4. State Management

Use `setState` judiciously: Only call `setState` when necessary to trigger a rebuild.

Consider **state management** solutions: For complex state management, explore options like `Provider`, `Riverpod` (builds on the `Provider` package), or `Business Logic Component (BLoC)`.

Performance Best Practices

5. Use Flutter's Build-in Widget

Use Flutter's optimized widgets whenever possible.

Find Out More

Widgets, <https://docs.flutter.dev/ui/widgets>

TextField, <https://docs.flutter.dev/cookbook/forms/text-input>

Layout, <https://docs.flutter.dev/ui/layout>